

**Lab Title: Data Carving with XXD,
Binwalk and Scalpel**

Student Name: Samuel Chinedu Onu

Student ID: 2025/FWSD/11518

**Course Name: SBT-DF202: Computer and
Digital Forensics**

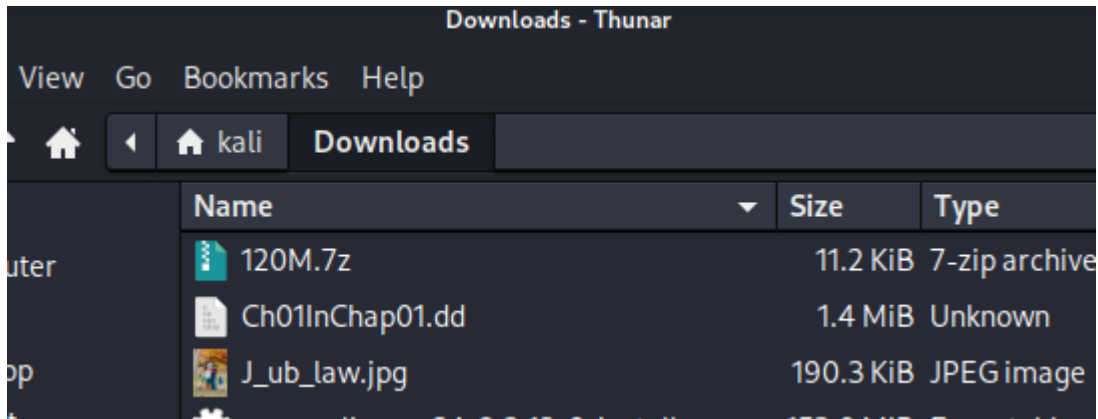
Instructor Name: Aminu Ibrahim Jalo

Date of Submission: 4th September 2026

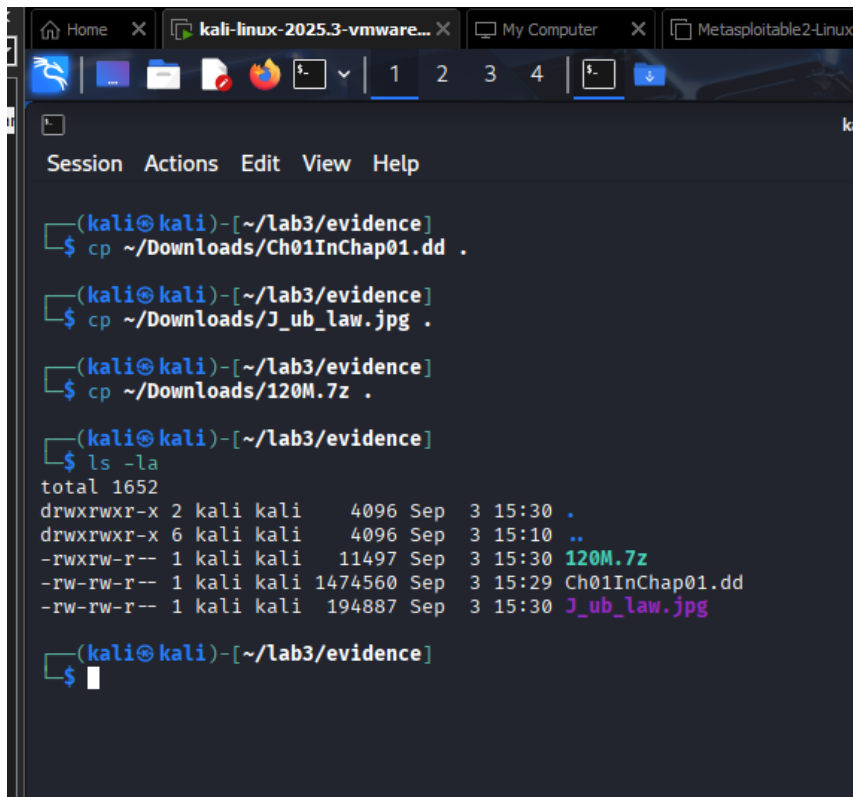
Computer and Digital Forensics

LAB 3: SBT-DF202: Data Carving with XXD, Binwalk and Scalpel

Files downloaded



Files moved to Lab3/evidence folder



Part A - File Signatures and Hex Analysis

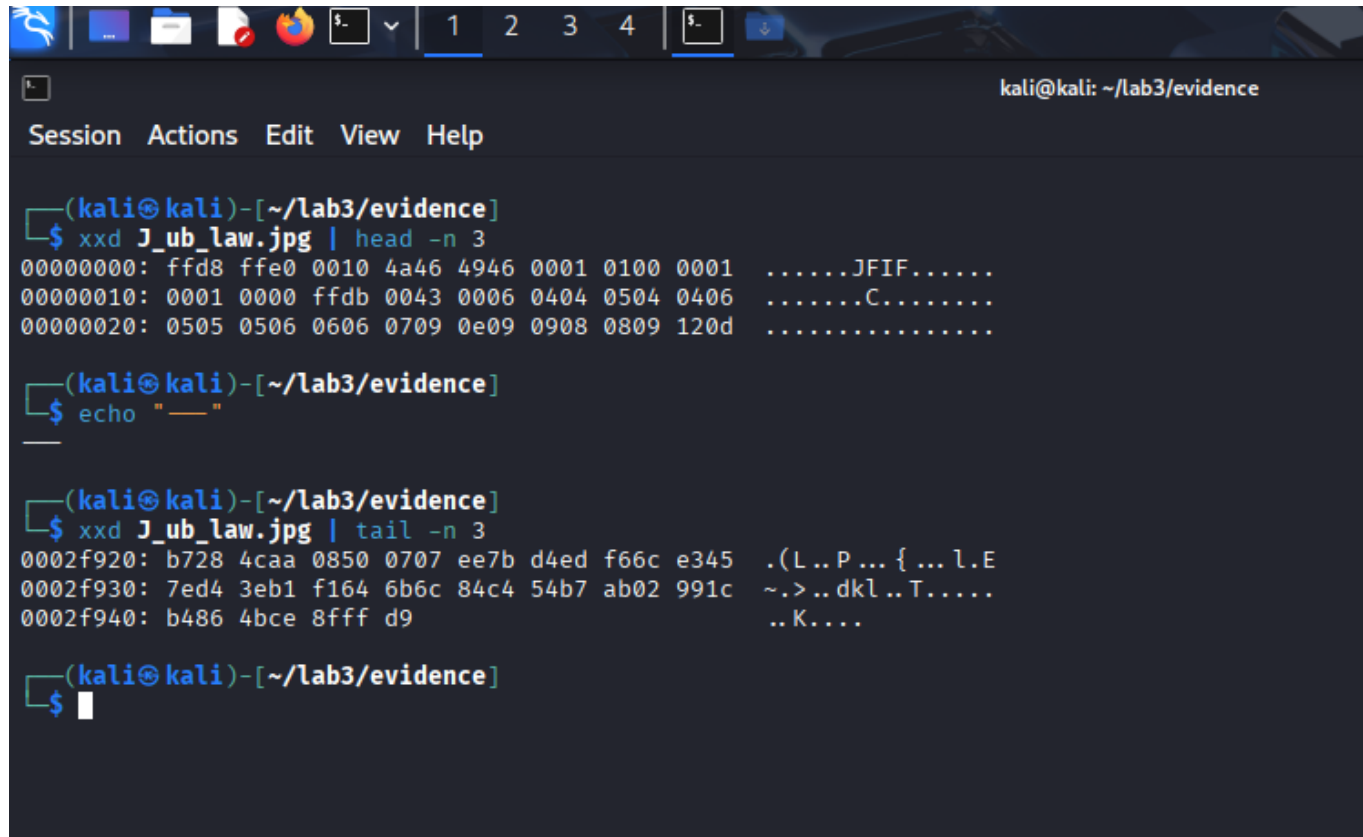
Generation of baseline hashes

```
(kali㉿kali)-[~/lab3/evidence]
$ cd ~/lab3/evidence

(kali㉿kali)-[~/lab3/evidence]
$ md5sum * | tee ../report/00_baseline_md5.txt
9f5004ea37b6605347d25da920098961 120M.7z
a117773bcf1fc88ec0ab8e0a349fbbcb Ch01InChap01.dd
16b1a4e8c11fe95d7d2e13097f195121 J_ub_law.jpg

(kali㉿kali)-[~/lab3/evidence]
$ sha256sum * | tee ../report/00_baseline_sha256.txt
5b9508e478d8c0f21bffd38b27aa164a04c27ae660ec5fc65c6276bdf1f5734 120M.7z
3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122 Ch01InChap01.dd
d9b8386f2c0baf9c42585a1d935bc481415532f1ce271afe031f173350c2b43b J_ub_law.jpg

(kali㉿kali)-[~/lab3/evidence]
$
```



```
(kali㉿kali)-[~/lab3/evidence]
$ xxd J_ub_law.jpg | head -n 3
00000000: ffd8 ffe0 0010 4a46 4946 0001 0100 0001  ....JFIF.....
00000010: 0001 0000 ffdb 0043 0006 0404 0504 0406  ....C.....
00000020: 0505 0506 0606 0709 0e09 0908 0809 120d  ....

(kali㉿kali)-[~/lab3/evidence]
$ echo "___"
___

(kali㉿kali)-[~/lab3/evidence]
$ xxd J_ub_law.jpg | tail -n 3
0002f920: b728 4caa 0850 0707 ee7b d4ed f66c e345  .(L..P...{...l.E
0002f930: 7ed4 3eb1 f164 6b6c 84c4 54b7 ab02 991c  ~.>..dkl..T.....
0002f940: b486 4bce 8fff d9  ..K....

(kali㉿kali)-[~/lab3/evidence]
$
```

SOI (Start of Image): ff d8 ff e0 at offset 00000000, followed by 4a46 4946 00 = ASCII JFIF\0 — confirms this is a standard JFIF-format JPEG (not Exif, which would show ff d8 ff e1).

EOI (End of Image): ff d9 at the very last two bytes, offset 0002f946

File length: 0x2f948 bytes = 194,888 bytes

```
kali@kali: ~/lab3/evidence
Session Actions Edit View Help

(kali@kali)-[~/lab3/evidence]
$ xxd J_ub_law.jpg | head -n 3
00000000: ffd8 ffe0 0010 4a46 4946 0001 0100 0001  ....JFIF.....
00000010: 0001 0000 ffd9 0043 0006 0404 0504 0406  ....C.....
00000020: 0505 0506 0606 0709 0e09 0908 0809 120d  ....

(kali@kali)-[~/lab3/evidence]
$ echo "___"
___

(kali@kali)-[~/lab3/evidence]
$ xxd J_ub_law.jpg | tail -n 3
0002f920: b728 4caa 0850 0707 ee7b d4ed f66c e345  .(L..P...{...l.E
0002f930: 7ed4 3eb1 f164 6b6c 84c4 54b7 ab02 991c  ~.>..dkl..T.....
0002f940: b486 4bce 8fff d9  ..K....

(kali@kali)-[~/lab3/evidence]
$
```

Creation of the plain hex dump and reconstruct the binary

```
(kali@kali)-[~/lab3/evidence]
$ xxd -p J_ub_law.jpg > J_ub_law_plain.hex

(kali@kali)-[~/lab3/evidence]
$ wc -l J_ub_law_plain.hex
6497 J_ub_law_plain.hex

(kali@kali)-[~/lab3/evidence]
$ head -c 200 J_ub_law_plain.hex
ffd8ffe000104a46494600010100000100010000ffdb0043000604040504
040605050506060607090e09080809120d0d0a0e151216161512141417
1a211c17181f1914141d271d1f2223252525161c292c28242b21242524ff
db004301060606090

(kali@kali)-[~/lab3/evidence]
$ echo

(kali@kali)-[~/lab3/evidence]
$ xxd -r -p J_ub_law_plain.hex J_ub_law_reconstructed.jpg

(kali@kali)-[~/lab3/evidence]
$ ls -la J_ub_law.jpg J_ub_law_reconstructed.jpg
-rw-rw-r-- 1 kali kali 194887 Sep  3 15:30 J_ub_law.jpg
-rw-rw-r-- 1 kali kali 194887 Sep  3 15:43 J_ub_law_reconstructed.jpg

(kali@kali)-[~/lab3/evidence]
$
```

Findings: xxd -p produced a continuous hex stream (6497 lines, 30 bytes/line by default) with no offset or ASCII columns, which is exactly what makes it reversible with xxd -r -p. The reconstructed file is byte-identical in size to the original (194,887 bytes).

```
(kali㉿kali)-[~/lab3/evidence]
$ xxd -p J_ub_law.jpg > J_ub_law_plain.hex

(kali㉿kali)-[~/lab3/evidence]
$ wc -l J_ub_law_plain.hex
6497 J_ub_law_plain.hex

(kali㉿kali)-[~/lab3/evidence]
$ head -c 200 J_ub_law_plain.hex
ffd8ffe000104a46494600010100000100010000ffdb0043000604040504
040605050506060607090e0909080809120d0d0a0e151216161512141417
1a211c17181f1914141d271d1f2223252525161c292c28242b21242524ff
db004301060606090

(kali㉿kali)-[~/lab3/evidence]
$ echo

(kali㉿kali)-[~/lab3/evidence]
$ xxd -r -p J_ub_law_plain.hex J_ub_law_reconstructed.jpg

(kali㉿kali)-[~/lab3/evidence]
$ ls -la J_ub_law.jpg J_ub_law_reconstructed.jpg
-rw-rw-r-- 1 kali kali 194887 Sep  3 15:30 J_ub_law.jpg
-rw-rw-r-- 1 kali kali 194887 Sep  3 15:43 J_ub_law_reconstructed.jpg

(kali㉿kali)-[~/lab3/evidence]
$
```

Verify the reconstruction with hashes

```
(kali㉿kali)-[~/lab3/evidence]
$ md5sum J_ub_law.jpg J_ub_law_reconstructed.jpg
16b1a4e8c11fe95d7d2e13097f195121 J_ub_law.jpg
16b1a4e8c11fe95d7d2e13097f195121 J_ub_law_reconstructed.jpg

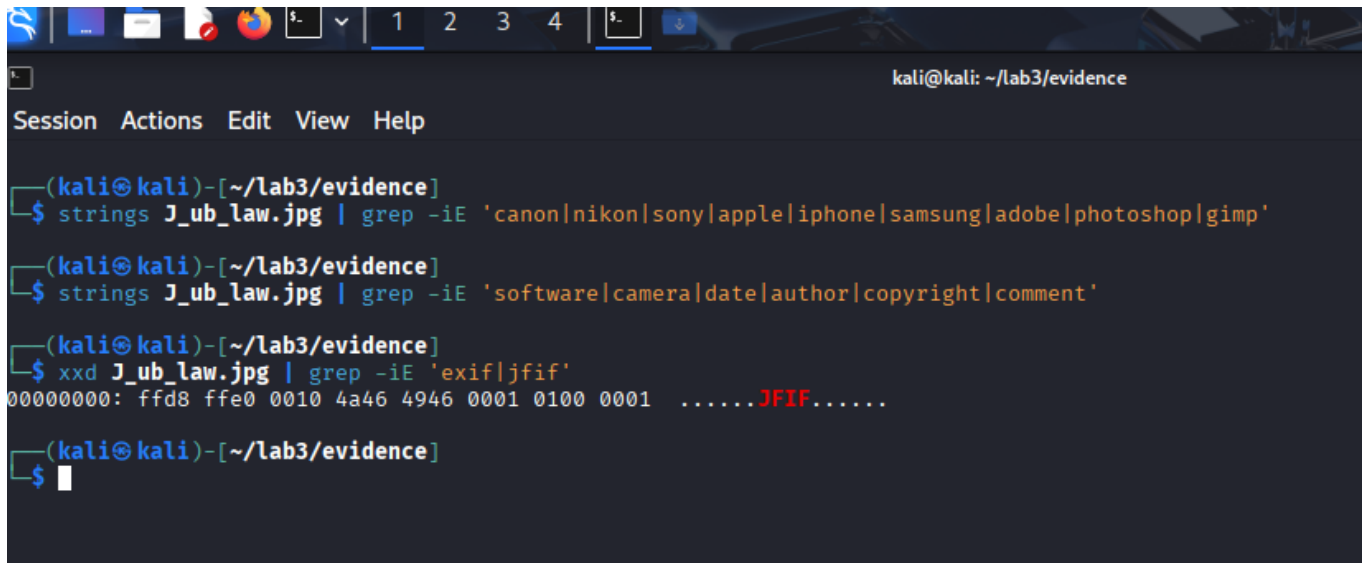
(kali㉿kali)-[~/lab3/evidence]
$ sha256sum J_ub_law.jpg J_ub_law_reconstructed.jpg
d9b8386f2c0baf9c42585a1d935bc481415532f1ce271afe031f173350c2b43b J_ub_law.jpg
d9b8386f2c0baf9c42585a1d935bc481415532f1ce271afe031f173350c2b43b J_ub_law_reconstructed.jpg

(kali㉿kali)-[~/lab3/evidence]
$
```

Finding: MD5 and SHA-256 hashes of the original and reconstructed files are identical, proving the hex-dump → reconstruction round trip is bit-for-bit lossless. This demonstrates why cryptographic hashing (not visual comparison or file size alone) is the forensic standard for verifying evidence integrity — any single-bit change

would have produced a completely different hash (the avalanche effect), so a match this exact is strong proof of authenticity.

Part B - Examine Embedded Metadata-Like Content

A terminal window on a Kali Linux system. The prompt is 'kali@kali: ~/lab3/evidence'. The user runs three commands: 1. 'strings J_ub_law.jpg | grep -iE 'canon|nikon|sony|apple|iphone|samsung|adobe|photoshop|gimp'' which returns no output. 2. 'strings J_ub_law.jpg | grep -iE 'software|camera|date|author|copyright|comment'' which also returns no output. 3. 'xxd J_ub_law.jpg | grep -iE 'exif|jifif'' which returns '00000000: ffd8 ffe0 0010 4a46 4946 0001 0100 0001JFIF.....'.

```
(kali@kali)-[~/lab3/evidence]
$ strings J_ub_law.jpg | grep -iE 'canon|nikon|sony|apple|iphone|samsung|adobe|photoshop|gimp'

(kali@kali)-[~/lab3/evidence]
$ strings J_ub_law.jpg | grep -iE 'software|camera|date|author|copyright|comment'

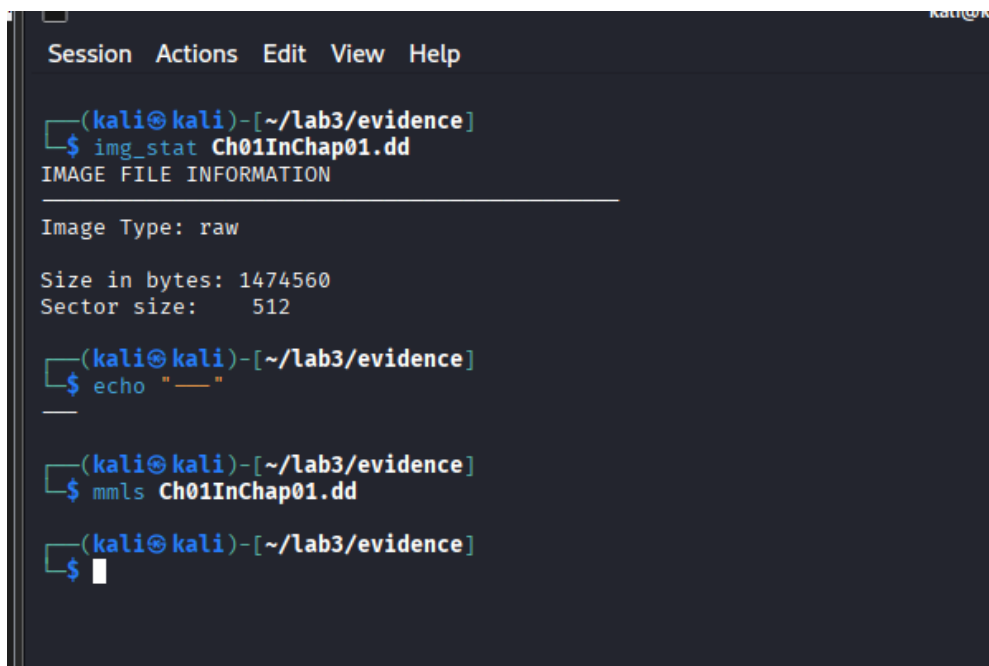
(kali@kali)-[~/lab3/evidence]
$ xxd J_ub_law.jpg | grep -iE 'exif|jifif'
00000000: ffd8 ffe0 0010 4a46 4946 0001 0100 0001 .....JFIF.....

(kali@kali)-[~/lab3/evidence]
$
```

Nothing was found for camera make and picture editing softwares.

Image format marker – **JFIF** was found.

Part C - Direct Data-Unit Examination with The Sleuth Kit

A terminal window on a Kali Linux system. The prompt is 'kali@kali: ~/lab3/evidence'. The user runs three commands: 1. 'img_stat Ch01InChap01.dd' which outputs 'IMAGE FILE INFORMATION' followed by 'Image Type: raw', 'Size in bytes: 1474560', and 'Sector size: 512'. 2. 'echo "——"' which outputs '——'. 3. 'mmls Ch01InChap01.dd' which returns no output.

```
(kali@kali)-[~/lab3/evidence]
$ img_stat Ch01InChap01.dd
IMAGE FILE INFORMATION
-----
Image Type: raw

Size in bytes: 1474560
Sector size: 512

(kali@kali)-[~/lab3/evidence]
$ echo "——"
——

(kali@kali)-[~/lab3/evidence]
$ mmls Ch01InChap01.dd

(kali@kali)-[~/lab3/evidence]
$
```

Finding: The image is exactly 1,474,560 bytes with 512-byte sectors

```

(kali@kali)-[~/lab3/evidence]
$ fsstat -o 0 Ch01InChap01.dd
FILE SYSTEM INFORMATION

File System Type: FAT12

OEM Name: , 'k0nIHC
Volume ID: 0x0
Volume Label (Boot Sector):
Volume Label (Root Directory):
File System Type Label: 3MM

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 2879
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 9
* FAT 1: 10 - 18
* Data Area: 19 - 2879
** Root Directory: 19 - 32
** Cluster Area: 33 - 2879

METADATA INFORMATION

Range: 2 - 45782
Root Directory: 2

CONTENT INFORMATION

Sector Size: 512
Cluster Size: 512
Total Cluster Range: 2 - 2848

FAT CONTENTS (in sectors)

33-236 (204) → EOF
285-311 (27) → EOF

(kali@kali)-[~/lab3/evidence]
$ █

```

Findings:

1. Confirmed filesystem type: FAT12, consistent with a 1.44MB floppy image.
2. Sector size = cluster size = 512 bytes (no multi-sector clusters on this small volume).
3. The FAT table shows two live allocation chains: sectors 33–236 (204 sectors) and sectors 285–311 (27 sectors), each terminating in EOF — meaning two separate files/chains are currently allocated in the data area.

```
(kali㉿kali)-[~/lab3/evidence]
$ fls -o 0 -r -p Ch01InChap01.dd
r/r 5: Client Info.mdb
r/r * 8: Billing Letter.doc
r/r * 11: confirmation.txt
r/r 13: Income.xls
r/r * 15: letter1.txt
r/r * 17: Regrets.doc
v/v 45779: $MBR
v/v 45780: $FAT1
v/v 45781: $FAT2
V/V 45782: $OrphanFiles

(kali㉿kali)-[~/lab3/evidence]
$
```

Findings:

1. Six real files exist in the FAT12 root directory, referenced by TSK's metadata (inode) addresses 5, 8, 11, 13, 15, 17.
2. Four are marked deleted (*): inode 8 (Billing Letter.doc), inode 11 (confirmation.txt), inode 15 (letter1.txt), and inode 17 (Regrets.doc).
3. Only inodes 5 (Client Info.mdb) and 13 (Income.xls) are currently allocated/live.

Examine “Billing Letter”

```
kali@kali: ~/lab3/evidence
Session Actions Edit View Help

(kali㉿kali)-[~/lab3/evidence]
$ istat -o 0 Ch01InChap01.dd 8
Directory Entry: 8
Not Allocated
File Attributes: File, Archive
Size: 24064
Name: _ILLIN-1.DOC

Directory Entry Times:
Written: 2005-12-09 06:50:28 (WAT)
Accessed: 2005-12-09 00:00:00 (WAT)
Created: 2005-12-09 06:59:05 (WAT)

Sectors:
237 238 239 240 241 242 243 244
245 246 247 248 249 250 251 252
253 254 255 256 257 258 259 260
261 262 263 264 265 266 267 268
269 270 271 272 273 274 275 276
277 278 279 280 281 282 283

(kali㉿kali)-[~/lab3/evidence]
$ echo "___"
___

(kali㉿kali)-[~/lab3/evidence]
$ icat -o 0 Ch01InChap01.dd 8 > ../output/inode8_billing_letter.doc

(kali㉿kali)-[~/lab3/evidence]
$ file ../output/inode8_billing_letter.doc
../output/inode8_billing_letter.doc: Composite Document File V2 Document, Little Endian, Os: Windows, Version 4.10, Code page: 1252, Title: 13 October 2003, Author: Amelia Phillips, Template: Normal.dot, Last Saved By: Amelia Phillips, Revision Number: 2, Name of Creating Application: Microsoft Word 9.0, Total Editing Time: 06:00, Create Time/Date: Sat Nov 23 03:06:00 2002, Last Saved Time/Date: Fri Dec 9 14:50:00 2005, Number of Pages: 1, Number of Words: 118, Number of Characters: 678, Security: 0

(kali㉿kali)-[~/lab3/evidence]
$ ls -la ../output/inode8_billing_letter.doc
-rw-rw-r-- 1 kali kali 24064 Sep 3 16:19 ../output/inode8_billing_letter.doc

(kali㉿kali)-[~/lab3/evidence]
$
```


Findings:

- `istat` confirms the directory entry for inode 8 is "**Not Allocated**" (i.e., marked deleted), original filename `_ILLIN~1.DOC` (the leading byte of "Billing" was overwritten with `0xE5` on deletion, FAT's standard delete marker — `fls` reconstructs the display name from the long-filename entry, while `istat` shows the raw short-name with the artifact of deletion, `_`, in place of the erased first character).

```
(kali㉿kali)-[~/lab3/evidence]
$ md5sum ../output/inode8_billing_letter.doc
9fe241d0dde27e83442010b3eee5ad32  ../output/inode8_billing_letter.doc

(kali㉿kali)-[~/lab3/evidence]
$ sha256sum ../output/inode8_billing_letter.doc
7eefeb4187bb551f366da33bcab5b7224f905bb98f37797c28f183f72aec2e99  ../output/inode8_billing_letter.doc

(kali㉿kali)-[~/lab3/evidence]
$
```

Hash of recovered "Billing_letter".